Assignment 3

In this assignment you will build a small network of virtual machines, configure its routing by hand, and implement TCP and HTTP yourself on top of raw sockets.

1 The Network (Part A)

In this part you set up four Linux virtual machines connected by four internal networks, and configure every address and route by hand so that traffic follows the paths and constraints below. You submit one configuration script per machine.

1.1 Topology

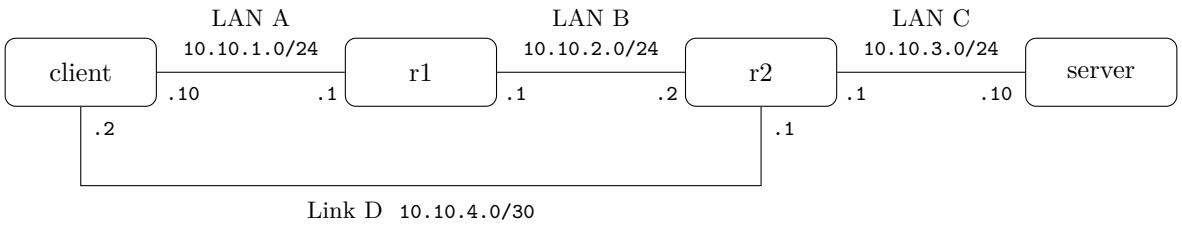


Figure 1: Network topology: four machines, LANs A–C and Link D.

A number such as `.10` next to a machine is the last part of that machine's address on that network; for example, `.10` next to the client on LAN A (`10.10.1.0/24`) means `10.10.1.10`. The full list is in Table 1.

1.2 Addressing Plan

Use exactly the addresses in Table 1.

Machine	Interface on	Address
client	LAN A	`10.10.1.10/24`
client	Link D	`10.10.4.2/30`
r1	LAN A	`10.10.1.1/24`
r1	LAN B	`10.10.2.1/24`
r2	LAN B	`10.10.2.2/24`
r2	LAN C	`10.10.3.1/24`
r2	Link D	`10.10.4.1/30`
server	LAN C	`10.10.3.10/24`

Table 1: Addressing plan.

1.3 Routing Requirements

A1. Every machine can reach every address on LANs A, B and C, and the client and r2 can reach each other over Link D.

A2. Packets from the client to the server travel client → r1 → r2 → server.

A3. Packets from the server to the client travel server → r2 → client, over Link D.

A4. r1 and r2 are routers; the client and the server are hosts and must not forward packets.

A5. r1 reaches all networks it is not attached to with a single route, and has no other routes besides its directly connected networks.

A6. r2 has no default route.

A7. All machines start with strict reverse-path filtering (`rp_filter` = 1), and it must stay strict on r1 and the server.

A8. Only static configuration: no DHCP and no routing daemons.

1.4 Network Configuration Scripts

Submit four bash scripts in `netcfg/`, each configuring one machine. Interface names are passed as arguments (Table 2):

Script	Usage
`netcfg/client.sh`	`client.sh IF_A IF_D`
`netcfg/r1.sh`	`r1.sh IF_A IF_B`
`netcfg/r2.sh`	`r2.sh IF_B IF_C IF_D`
`netcfg/server.sh`	`server.sh IF_C`

Table 2: Network configuration scripts.

- Each script runs once, as root, on a freshly booted machine whose interfaces exist but are down and have no addresses, and must leave the machine meeting all the requirements above.
- Use only `ip` and `sysctl`.
- Refer to interfaces only through the arguments, and do not touch any other interface.

The configuration does not need to survive a reboot.

1.5 Virtual Machines

Build the network with four Ubuntu VMs (22.04 preferred, or a later release) in VirtualBox (or any hypervisor that supports isolated internal networks, such as VMware Fusion or UTM on Apple Silicon Macs). LANs A, B, C and Link D are four separate internal networks. Once your packages are installed, each VM must be attached only to its internal networks (a NAT adapter may be used temporarily for installing software). The internal networks must be isolated: no DHCP server and no connection to the host computer.

Before running your scripts, set `rp_filter` to 1 on every interface of every VM (Ubuntu's default is 2), since that is the state your scripts are evaluated from.

2 TCP/IPv4 over Raw Sockets (Part B)

In this part you implement TCP over raw sockets: you build every IPv4 and TCP header you send, parse every header you receive, and handle connection setup, reliable data transfer and teardown yourself. Your HTTP client and server both use this implementation. Requirements T1–T14 apply to your client and server programs; r1 and r2 only forward packets, which the kernel does for them.

2.1 Outgoing Packets

T1. Build the complete IPv4 header yourself, including its checksum. The identification field must be non-zero.

T2. Build the complete TCP header yourself, including its checksum.

T3. SYN and SYN-ACK segments carry exactly one option, MSS = 1460. No other options are sent. Data segments carry at most min(1460, peer's MSS) bytes of payload.

2.2 Incoming Packets

T4. Verify the checksums of every received packet and silently drop invalid ones. Handle TCP options: use the peer's MSS and ignore the others.

T5. Process only packets that belong to your connection (matching addresses and ports).

2.3 Connection Management

T6. Choose a random initial sequence number.

T7. Three-way handshake. An unanswered SYN or SYN-ACK is retransmitted after at most 1 s, and the handshake keeps trying for at least 6 s before giving up.

T8. Abort the connection immediately when a RST arrives for it.

T9. Teardown. Acknowledge the peer's FIN. After sending your FIN, retransmit it until it is acknowledged and wait for the peer's FIN, but give up after at most 2 s. TIME_WAIT is not required.

2.4 Reliable Data Transfer

T10. Sending. Never have more unacknowledged data than both your congestion window (T14) and the peer's advertised window allow. Handle cumulative ACKs, and retransmit lost data (retransmission timeout of at most 1 s). Give up if nothing is acknowledged for 10 s.

T11. Receiving. Deliver bytes to HTTP in order and exactly once. Send a cumulative ACK for every data segment received, including duplicate and out-of-order ones. Advertise a fixed window of 65535.

T12. Loss and reordering. Transfers must complete correctly under packet loss and reordering; out-of-order segments inside your window must be kept, not discarded.

T13. Ports. The client uses a random source port in 61000–65535. The server listens on the port given by `-p`.

2.5 Congestion Control

T14. Implement congestion control in the style of standard TCP (for example Reno): your sending rate must grow while the network has spare capacity and back off when packets are lost. When your client downloads from your server while other TCP connections (for example `curl` downloading from a standard web server) share the same bottleneck link, the connections should share the link roughly equally. This must also hold with several connections of each kind at the same time.

2.6 Packet Log

With `--log FILE`, both programs write one line per TCP segment they send (including retransmissions and pure ACKs) or receive and accept (belongs to the connection and passes both checksums), in exactly this format, with single spaces (fields in Table 3):

```
DIR SRC_IP:SPORT > DST_IP:DPORT ttl=T id=I seq=S ack=A flags=0xFF win=W len=L mss=M
```

Field	Meaning
`DIR`	`SEND` or `RECV`
`ttl`, `id`	IPv4 TTL and identification, in decimal
`seq`, `ack`	raw 32-bit sequence and acknowledgment fields, unsigned decimal (not relative; print `ack` even when the ACK flag is off)
`flags`	byte 13 of the TCP header as `0x` and two lowercase hex digits, e.g. SYN `0x02`, SYN-ACK `0x12`, ACK `0x10`
`win`	raw 16-bit window field, in decimal
`len`	TCP payload length in bytes
`mss`	MSS option value if present, otherwise `-`

Table 3: Fields of a packet-log line.

Example (client side of a handshake; the numbers are illustrative, real sequence numbers are random 32-bit values):

```
SEND 10.10.1.10:62425 > 10.10.3.10:8000 ttl=64 id=301 seq=1000 ack=0 flags=0x02 win=65535 len=0 mss=1460
RECV 10.10.3.10:8000 > 10.10.1.10:62425 ttl=62 id=0 seq=5000 ack=1001 flags=0x12 win=64240 len=0 mss=1460
SEND 10.10.1.10:62425 > 10.10.3.10:8000 ttl=64 id=302 seq=1001 ack=5001 flags=0x10 win=65535 len=0 mss=-
```

`SEND` lines contain the values you put into the packet, and `RECV` lines the values you parsed. Flush the log after every line.

2.7 Not Required

Window scaling, SACK, timestamps; TIME_WAIT; urgent data; IP options; fragmentation; IPv6; zero-window probing; more than one connection at a time; HTTP keep-alive; HTTPS; redirects; compression; request bodies.

3 HTTP Client (Part C)

In this part you write an HTTP client that uses your TCP to download one URL from a standard web server and save the response body.

```
sudo ./client/run-client URL [-o OUTFILE] [--log LOGFILE]
```

- URL: `http://HOST[:PORT][/PATH]`, where HOST is an IPv4 address or a host name. The default port is 80 and the default path is `/`. PATH may include a query string (e.g. `/search?q=tcp`).
- Request: exactly one HTTP/1.1 `GET` per run. The request target is PATH exactly as it appears in the URL (including any query string), the `Host` header is exactly as written in the URL, the request includes `Connection: close`, and it does not include `Accept-Encoding`.
- Response framing: support `Content-Length`, `Transfer-Encoding: chunked` (ignoring chunk extensions and trailers), and bodies that end when the server closes the connection. Header names are case-insensitive.
- Output: write only the body to OUTFILE, or to stdout if `-o` is not given. Print the status line to stderr.
- Exit code: `0` for a complete response with a 2xx status; `1` for a complete response with any other status (still save the body; do not follow redirects); `2` for anything else (invalid URL, DNS failure, connection refused, timeout, incomplete body).
- Close the connection (T9) once the response is complete.

4 HTTP Server (Part D)

In this part you write an HTTP server that uses your TCP to serve files from a directory to standard clients such as `curl` and `wget`.

```
sudo ./server/run-server [-p PORT] [-d DOCROOT] [--log LOGFILE]
```

The defaults are `-p 8080` and `-d ./www`.

- The server runs until killed, handles one connection at a time, and never exits because of a misbehaving client.
- It accepts connections on PORT for any of the machine's addresses and replies from the address the SYN was sent to.
- For each connection it reads the request head, sends one response, and closes the connection (T9). If the request head is not complete within 10 s, it closes the connection without a response.

- It chooses the response as in Table 4.

#	Condition	Status
1	The request line is not `METHOD SP TARGET SP HTTP/x.y`, or TARGET does not start with `/`	`400 Bad Request`
2	METHOD is neither `GET` nor `HEAD`	`501 Not Implemented`
3	A path segment of TARGET is `..`	`403 Forbidden`
4	The file does not exist	`404 Not Found`
5	Otherwise	`200 OK`

Table 4: Choosing the server's response (checked in this order).

- Mapping: ignore any query string (everything from `?` onwards) and append the remaining path to DOCROOT. A path ending in `/`, or naming a directory, maps to `index.html` in it. Percent-decoding is not required.
- Every response, including errors, is `HTTP/1.1` with correct `Content-Length`, `Content-Type` and `Connection: close` headers. Error responses carry a small HTML body.
- A HEAD response has the same headers as GET would, and no body.
- Content-Type by file extension, case-insensitive (Table 5).

Extension	Content-Type	Extension	Content-Type
`.html`, `.htm`	`text/html`	`.png`	`image/png`
`.txt`	`text/plain`	`.jpg`, `.jpeg`	`image/jpeg`
`.css`	`text/css`	`.gif`	`image/gif`
`.js`	`application/javascript`	`.pdf`	`application/pdf`
`.json`	`application/json`	anything else	`application/octet-stream`

Table 5: Content-Type by file extension.

5 Client–Server Interoperation (Part E)

Your client on the client machine must download files from your server on the server machine. (Client and server must always run on different machines; loopback is not supported.)

6 Implementation Requirements

This section lists the languages and rules for your programs, and how they are started.

6.1 Language

C, C++ or Python, using only the standard libraries. Your code must build and run on Ubuntu 22.04 (x86-64, gcc 11, Python 3.10) without Internet access; if you develop on a later release, do not use newer language features.

6.2 Raw Sockets

Your programs must send and receive all TCP traffic through raw IP sockets, and must supply the complete IPv4 header of every packet they send themselves: the socket must not add an IP header for you. Letting the kernel build the IP header (a raw socket without `IP_HDRINCL`) results in zero marks for Part B.

6.3 Not Allowed

- Kernel TCP: `SOCK_STREAM` sockets, `connect()` on a TCP socket, `listen()`, `accept()`.
- `AF_PACKET` sockets.
- Packet or HTTP libraries (e.g. scapy, dpkt, libpcap, libnet, `http.client`, `http.server`, `urllib`, `requests`, `socketserver`, `asyncio`, libcurl).
- Running other programs (`system()`, `popen()`, `subprocess`, `exec*()`), except in the launchers.

Using anything in this list results in zero marks for Parts B–E. Debugging tools (Wireshark, tcpdump, scapy, ...) may be used outside your submitted code.

6.4 Launchers

`client/run-client` starts your HTTP client and `server/run-server` your HTTP server. They must be executable, use `exec`, pass on all arguments, and work from any working directory. For example, a Python client's launcher can be:

```
#!/bin/sh
exec python3 "$(dirname "$0")/../src/myclient.py" "$@"
```

7 Submission

7.1 Submission File

Submit one ZIP file on Gradescope, named with the entry numbers of both team members, sorted alphabetically, exactly as they appear on the LMS, in capitals, without spaces or hyphens:

`A3_<entry_number1>_<entry_number2>.zip`

Only one team member uploads it to Gradescope and adds the other as a group member. The ZIP must have this structure; everything except the `Makefile` is mandatory:

```
A3_<entry_number1>_<entry_number2>.zip/
+-- netcfg/
|   +-- client.sh
|   +-- r1.sh
|   +-- r2.sh
|   +-- server.sh
+-- client/
|   +-- run-client
+-- server/
|   +-- run-server
+-- src/
|   +-- <your source code>
+-- Makefile (optional)
+-- README.md
+-- report.pdf
```

If a `Makefile` is present, `make` is run in the top-level directory (as root, without Internet access) before your programs are used. Do not include binaries, captures or VM images.

The `README.md` states the team members, the language and compiler or runtime, and how to build.

8 Report

Submit `report.pdf` with the following. Every screenshot must be taken on your four VMs (not in a simulated network), be readable, and show the command that produced it.

R1. The network-adapter settings of all four VMs, and `ip -br addr` and `ip route` on each.

R2. `traceroute -n` from client to server and from server to client, with an explanation of every hop: why the two directions differ, and why r2 appears with the address it does.

R3. The reverse-path filtering setting you chose on each machine, and why.

R4. Your client downloading an HTML page and a binary file of at least 100 KB from a standard web server on the server VM (e.g. `python3 -m http.server`), with the exit codes; the `md5sum` of each file must match that of the same URL fetched with `curl`.

R5. `curl`, `wget` and your client downloading the same file from your server; all three `md5sum`s must match the original.

R6. A Wireshark or `tcpdump -nn` capture of one complete connection of your client, and a short description of your retransmission, out-of-order buffering and congestion control.

9 Grading

Component	Weight
Implementation (Parts A–E)	70%
Part A: The Network	10%
Part B: TCP/IPv4 over Raw Sockets	15%
Part C: HTTP Client	15%
Part D: HTTP Server	15%
Part E: Client–Server Interoperation	15%
Report	10%
Viva	20%
Total	100%

Table 6: Grading.